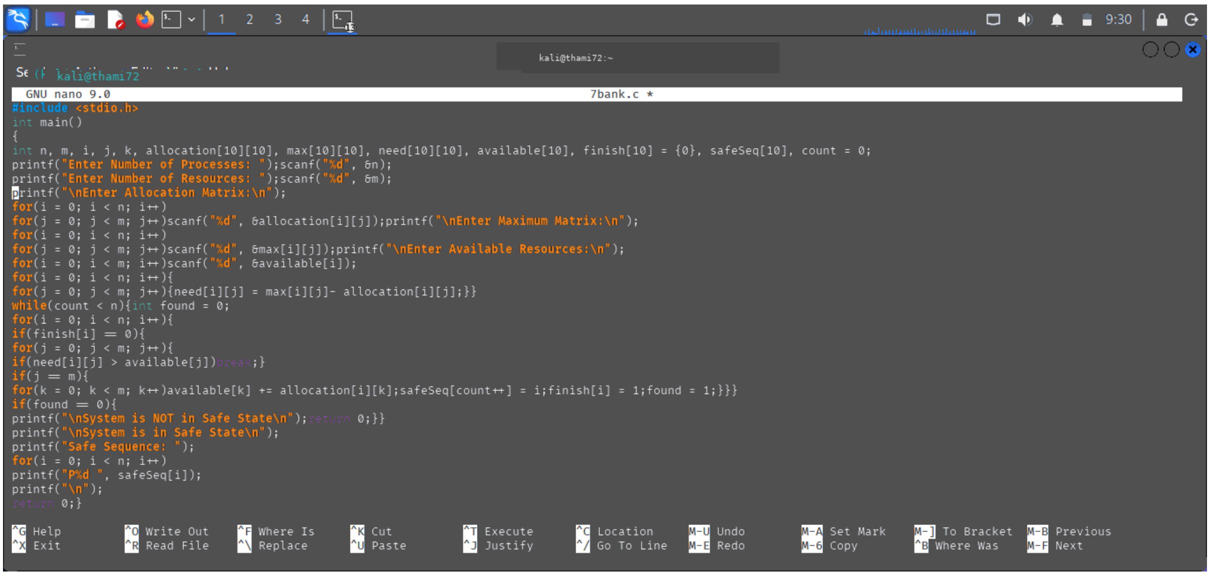
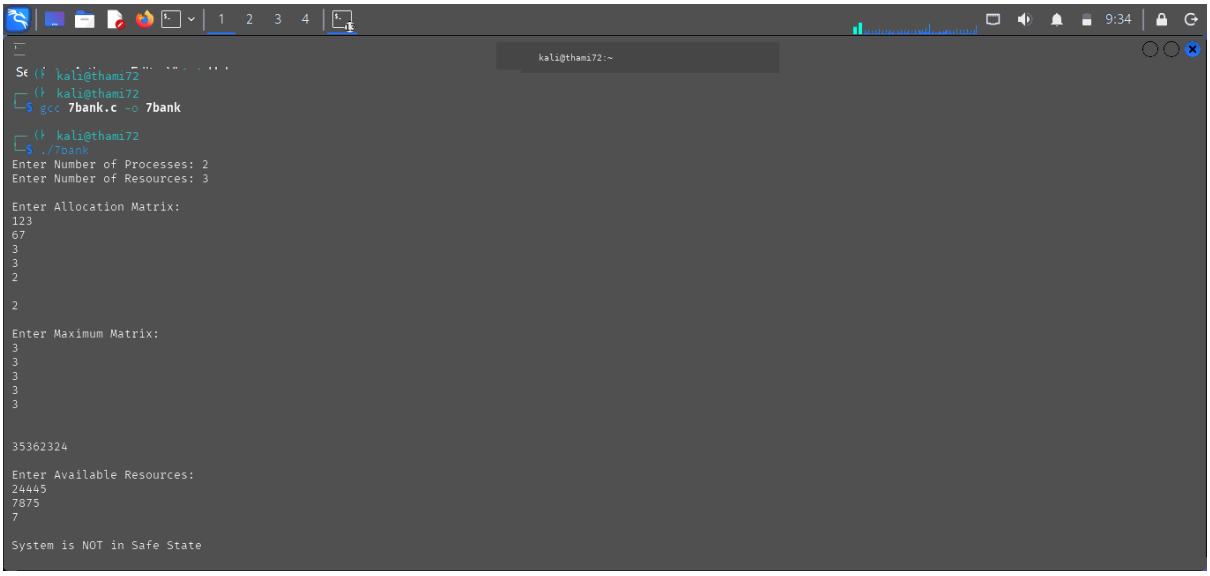
C PROGRAMMING :



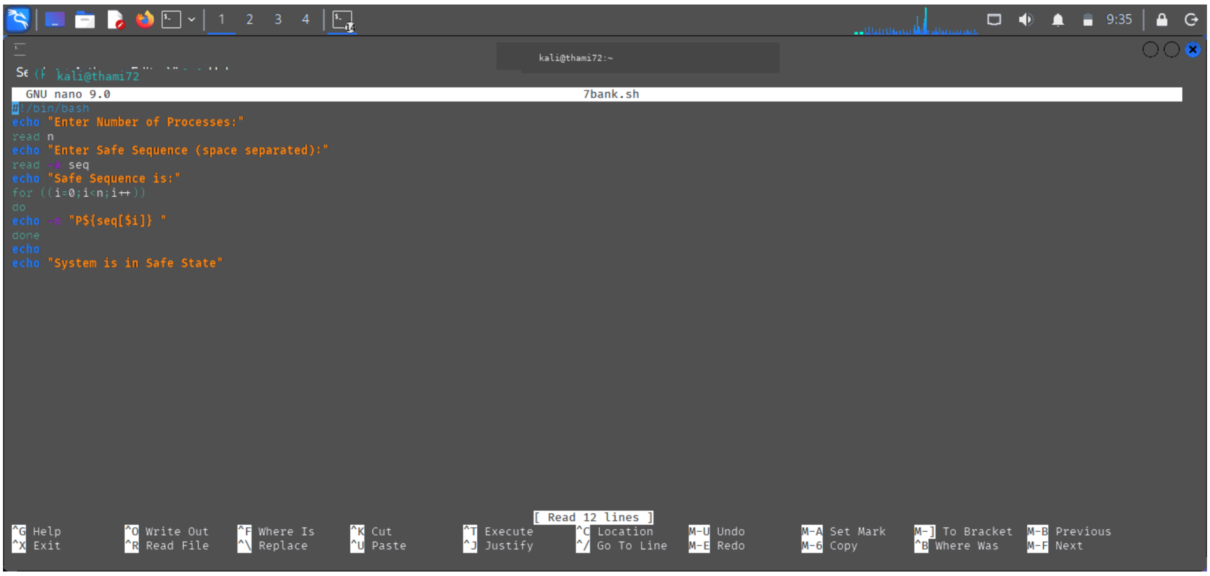
```
GNU nano 9.0                                   7bank.c *
#include <stdio.h>
int main()
{
int n, m, i, j, k, allocation[10][10], max[10][10], need[10][10], available[10], finish[10] = {0}, safeSeq[10], count = 0;
printf("Enter Number of Processes: ");scanf("%d", &n);
printf("Enter Number of Resources: ");scanf("%d", &m);
printf("\nEnter Allocation Matrix:\n");
for(i = 0; i < n; i++)
for(j = 0; j < m; j++)scanf("%d", &allocation[i][j]);printf("\nEnter Maximum Matrix:\n");
for(i = 0; i < n; i++)
for(j = 0; j < m; j++)scanf("%d", &max[i][j]);printf("\nEnter Available Resources:\n");
for(i = 0; i < m; i++)scanf("%d", &available[i]);
for(i = 0; i < n; i++){
for(j = 0; j < m; j++){need[i][j] = max[i][j]- allocation[i][j];}}
while(count < n){int found = 0;
for(i = 0; i < n; i++){
if(finish[i] == 0){
for(j = 0; j < m; j++){
if(need[i][j] > available[j])break;}
if(j == m){
for(k = 0; k < m; k++)available[k] += allocation[i][k];safeSeq[count++] = i;finish[i] = 1;found = 1;}}}
if(found == 0){
printf("\nSystem is NOT in Safe State\n");return 0;}}
printf("\nSystem is in Safe State\n");
printf("Safe Sequence: ");
for(i = 0; i < n; i++)
printf("P%d ", safeSeq[i]);
printf("\n");
return 0;}

^G Help      ^O Write Out   ^F Where Is   ^K Cut     ^T Execute   ^C Location     M-U Undo   M-A Set Mark   M-] To Bracket   M-B Previous
^X Exit      ^R Read File   ^\ Replace    ^U Paste   ^J Justify   ^/ Go To Line   M-E Redo   M-6 Copy       ^B Where Was     M-F Next
```

OUTPUT :



```
kali@thami72
$ gcc 7bank.c -o 7bank

kali@thami72
$ ./7bank
Enter Number of Processes: 2
Enter Number of Resources: 3

Enter Allocation Matrix:
123
67
3
3
2

2

Enter Maximum Matrix:
3
3
3
3
3


35362324

Enter Available Resources:
24445
7875
7

System is NOT in Safe State
```

SHELL PROGRAMMING :



OUTPUT :